

0.43.2. Plus lentement : vitesse 500 — doublé, deux positions

Les deux carrés dans le même programme : le A autour de (10, 8), puis le B autour de (4, 4).

1. Ce qu'il faut comprendre

C'est le 0.43 et le 0.43.1 réunis : deux lignes, deux lettres, deux positions.

Ce qui est nouveau ici : deux carrés dans le même programme. Le A tourne autour de (10, 8), comme au 0.43 ; puis le B autour de (4, 4), comme au 0.43.1.

L'un après l'autre : `carre` attend la fin de son carré avant de rendre la main. Le B ne commence que quand le A est revenu à son centre.

Les deux carrés ne se touchent pas : chacun a son coin de l'écran. À la fin, on voit les deux lettres, chacune à son centre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction `carre\(\)` — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1380 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
carre()         440 o ← main() - une fonction fournie par le langage
main()          69 o ← démarrage - ta fonction
VBlank          8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases     12 o ← effacer() - efface des cases
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()    × 4    52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()   × 4    12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :
carre() x 2 62 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

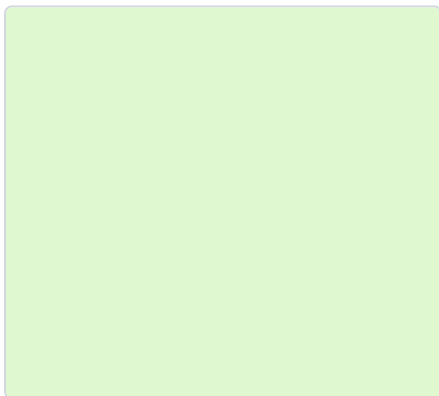
```
#include <carre> // une tuile qui tourne en carré  
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    // 1. Le A, autour de (10, 8) : le 0.43.  
    carre(10, 8, ALPHABET[0], 2, 1, 500, 1);  
  
    // 2. Le B, autour de (4, 4) : le 0.43.1.  
    // Il commence quand le A a fini : carre attend la fin de son carré.  
    carre(4, 4, ALPHABET[1], 2, 1, 500, 1);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

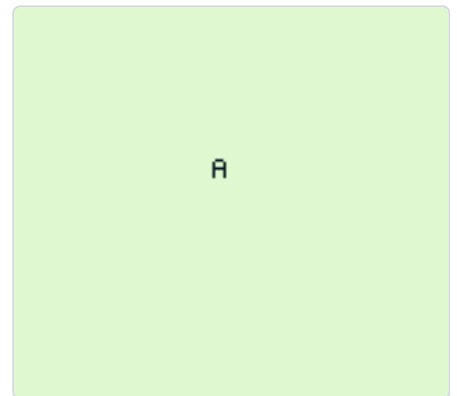
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



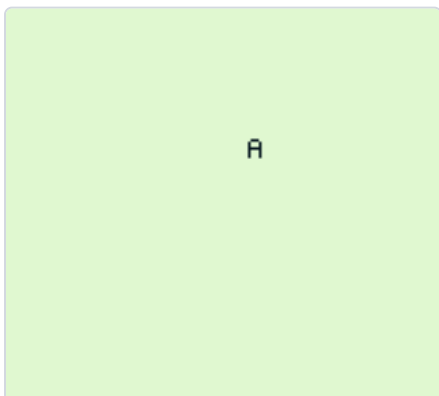
la toute première image — la console vient de démarrer — image 1



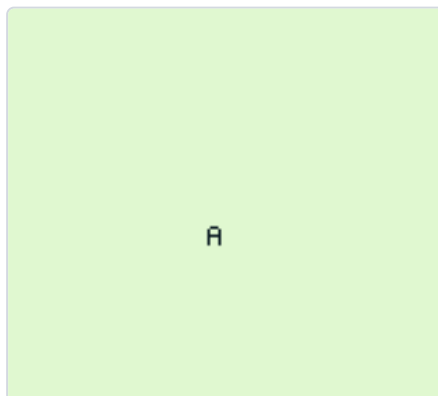
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A fait son carré autour de (10, 8), puis le B fait le même autour de (4, 4).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A fait son tour autour de (10, 8) : coin (12, 10)

✓ puis le B autour de (4, 4) : coin (2, 2)

✓ le B revient à son centre (4, 4) (4,4)

Ensuite : **0.44. Plus vite : vitesse 100** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++